

Smart India Hackathon 2026

User Manual

BeamLock

AI-Based Virtual Camera Tracking System for
Coarse Alignment of Mobile FSOC Terminals

Software installation, GUI, operation, and parameter configuration

Problem Statement ID: SIH26169

Organization: Department of Space / ISRO

Team: Cadets

Package: fsoc-virtual-tracker

Repository: <https://github.com/MudithDShetty/SIH>

Version 1.0 — September 11, 2026

Companion to the SIH Technical Report ([docs/sih-technical-report.pdf](#)).

Contents

1	Purpose of this Manual	2
2	System Requirements	2
3	Installation	2
3.1	Clone or unpack the project	2
3.2	Create a virtual environment	2
3.3	Install dependencies	2
3.4	Optional: CUDA PyTorch (NVIDIA GPU)	3
3.5	AI detector weights	3
3.6	Verify installation	3
3.7	Optional: standalone Windows executable	3
4	Launching the Application	3
4.1	Physics profile (default)	3
4.2	PS compliance profile	3
4.3	Exit	4
5	GUI Description	4
5.1	Main view (viewport)	5
5.2	Overlays on the camera feed	5
5.3	Control panel (bottom strip)	5
6	Application Operation	6
6.1	Typical demo flow (recommended)	6
6.2	Tracking states	7
6.3	Fine pointing handoff (automatic)	7
7	Parameter Configuration	7
7.1	Scenario presets	7
7.2	Live disturbance and plant parameters	7
7.3	Keyboard reference	8
8	Logging and Performance Report	8
8.1	Automatic logs	8
8.2	Streamlit report	8
8.3	Matched Classical vs AI benchmark	8
8.4	Benchmark-2 (judge MP4, PTZ bypass)	8
8.5	Other evaluation utilities	9
9	Troubleshooting	9

10 Related Documents	9
-----------------------------	----------

1 Purpose of this Manual

This user manual accompanies the BeamLock software deliverable for SIH26169. It describes how to install the application, launch Physics and PS-compliance profiles, operate the graphical user interface (GUI), configure disturbances and tracking parameters, interpret HUD overlays, generate performance logs, and run supporting evaluation tools. No specialized optical hardware is required; a laptop with Python 3.10+ is sufficient.

2 System Requirements

Table 1: Minimum / recommended environment

Item	Requirement
Operating system	Windows 10/11 (primary), Linux, or macOS
Python	3.10 or newer
RAM	8 GB minimum; 16 GB recommended for AI inference
Display	1280 × 900 or larger for Physics mode window
Disk	~2 GB free (venv + Ultralytics/Torch); more if building EXE
GPU (optional)	NVIDIA CUDA for faster YOLO training/inference

3 Installation

3.1 Clone or unpack the project

Place the repository so that the working directory is `fsoc-virtual-tracker/` (the folder containing `main.py`).

3.2 Create a virtual environment

```
cd fsoc-virtual-tracker
python -m venv .venv
```

Windows (PowerShell / cmd):

```
.venv\Scripts\activate
```

Linux / macOS:

```
source .venv/bin/activate
```

3.3 Install dependencies

```
pip install -r requirements.txt
```

Core packages include Pygame, NumPy, OpenCV, FilterPy, Ultralytics (YOLO), Pandas, Streamlit, and Matplotlib.

3.4 Optional: CUDA PyTorch (NVIDIA GPU)

If a CUDA GPU is available and Ultralytics installed a CPU-only Torch build:

```
pip uninstall -y torch torchvision
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
python -c "import torch; print(torch.cuda.is_available())"
```

3.5 AI detector weights

The live **AI** detector path expects trained weights (typically `weights/beacon_yolov8n.pt`). If weights are already shipped in the submission package, skip training. Otherwise generate synthetic data and train once:

```
python scripts/generate_training_data.py --num-frames 2400
python scripts/train.py --epochs 50 --imgsz 320
```

Without weights, selecting AI may fall back to Classical; Classical always remains available.

3.6 Verify installation

```
python scripts/smoke_test.py
```

A successful run prints that all smoke tests passed.

3.7 Optional: standalone Windows executable

```
pip install pyinstaller
python scripts/build_exe.py
```

Output appears under `dist/FS0CVirtualTracker/`. The bundle is large (Torch/Ultralytics); prefer the Python venv for size-constrained demos.

4 Launching the Application

4.1 Physics profile (default)

```
python main.py
```

or equivalently `python main.py --profile physics`. This is the BeamLock sandbox: IZN-1 class ~ 2.5 mrad FOV, 60 fps target, Gaussian beacon appearance.

4.2 PS compliance profile

```
python main.py --profile ps
```

Aligns screen/camera/FOV/slew/beacon geometry with the SIH26169 Parameters table (2000×2000 screen, 640×480 camera, $4^\circ \times 3^\circ$ FOV, $\sim 5^\circ/\text{s}$ slew, square beacon).

Table 2: Runtime profiles

Profile	Command	Purpose
Physics	<code>python main.py</code>	Optical-intuition sandbox + channel demos
PS	<code>python main.py --profile ps</code>	Parameters-table evaluation mode

4.3 Exit

Press **ESC** or close the window. Session CSV and summary JSON are written under `logs/` when the run ends (allow at least ~ 1 s of runtime before quitting).

5 GUI Description

The main window comprises a simulation viewport (overview + camera FOV) and a bottom control panel.

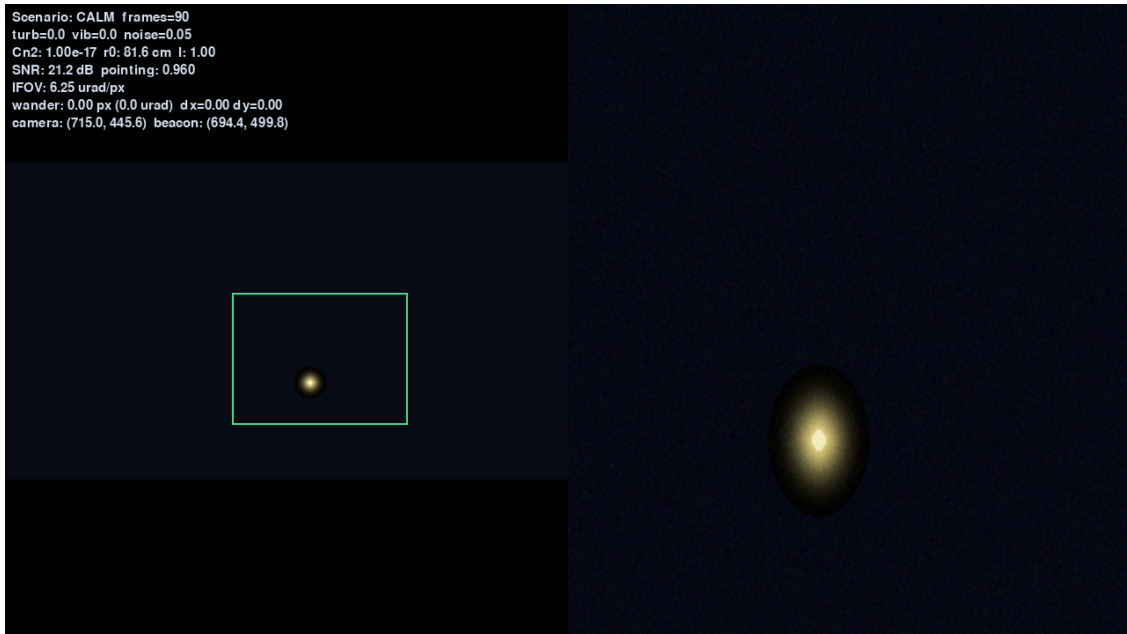


Figure 1: Representative GUI layout under the Calm scenario (overview, FOV, HUD).

5.1 Main view (viewport)

Table 3: Viewport regions

Region		Content
Scene overview		Full virtual world with moving beacon(s); green box marks camera FOV.
Camera FOV panel		Disturbed live crop seen by the detector (turbulence, vibration, noise, weather).
HUD (typically TL)		Camera boresight, detector mode, FPS, tracking error (px + μ rad), lock state.
Link gauge	Readiness	Score $S \in [0, 1]$; green ≥ 0.7 , yellow mid-band, red low / LOST.

5.2 Overlays on the camera feed

Table 4: Detection / tracking overlays

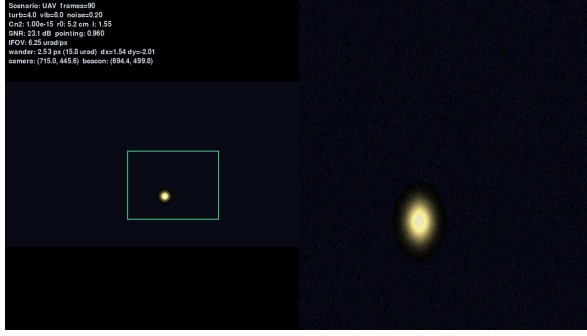
Symbol	Meaning
Cyan crosshair	Raw detector measurement
Magenta marker	Kalman filter estimate (camera is driven by this)
Search path (overview)	Re-acquisition waypoints when tracking state is LOST

5.3 Control panel (bottom strip)

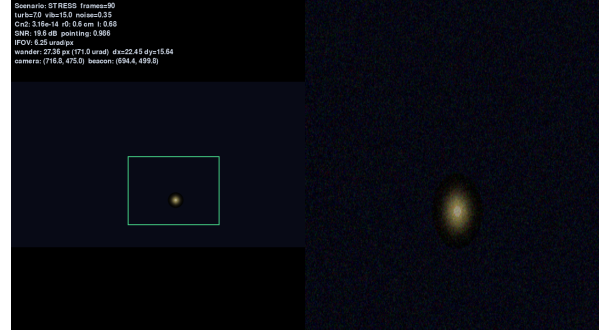
Mouse-driven controls (Physics window; PS window scales similarly):

Table 5: Control panel widgets

Control	Function
Cn2 proxy slider	Turbulence strength $s \in [0, 10]$ (log-mapped to Cn^2).
Vibration slider	Platform tip/tilt amplitude.
Sensor Noise slider	Camera noise level (0–1 class).
Slew Rate slider	Camera plant slew authority (px/s class; profile-dependent).
Trajectory buttons	Linear / Circular / Random / Fig-8.
Detector buttons	Classical (threshold centroid) / AI (hybrid fusion path).
Beacons buttons	1 or 2 beacons (second beacon acts as distractor).
Scenario buttons	Calm / UAV / Stress one-click presets.



(a) UAV preset



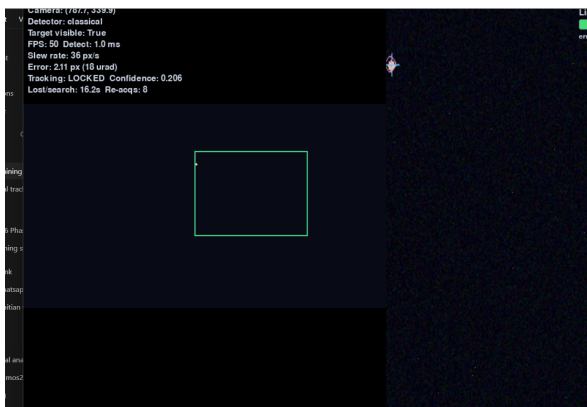
(b) Stress preset

Figure 2: GUI under mobile and stress disturbance presets.

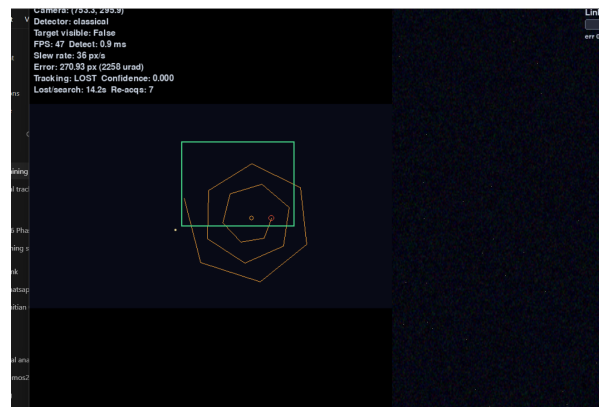
6 Application Operation

6.1 Typical demo flow (recommended)

1. Launch `python main.py`.
2. Click **Calm** (or press F1). Confirm LOCKED and low residual on HUD.
3. Switch to **UAV** (F2). Observe lock retention under moderate channel + vibration.
4. Switch to **Stress** (F3). Expect possible LOST → raster re-acquisition → re-lock.
5. Press **L** to blind the detector briefly and demonstrate re-acquisition deliberately.
6. Press **T** to toggle Classical ↔ AI (hybrid) and compare behavior.
7. Optionally press **B** for two beacons; press **4** for figure-of-eight motion.
8. Quit with ESC; open the generated CSV in Streamlit (Section 8).



(a) Re-acquisition after lock loss



(b) After forced detector blind (L)

Figure 3: Operational captures for lock-loss and recovery demos.

6.2 Tracking states

Table 6: Tracker state machine (visible on HUD / logs)

State	Meaning
LOCKED	Valid detection accepted this frame
COASTING	Temporary misses; filter coasts on prediction
LOST	Extended misses; raster re-acquisition owns the slew

6.3 Fine pointing handoff (automatic)

When Link Readiness $S \geq 0.7$, tracking is LOCKED, and residual stays inside the capture cone for consecutive frames, the Fine mock (4-QD / FSM) may engage. It exits with hysteresis when $S \leq 0.55$ or residual grows. This is a demonstration of coarse-to-fine handoff, not a flight FSM calibration.

7 Parameter Configuration

7.1 Scenario presets

Table 7: Built-in disturbance presets

Preset	Turb	Vib	Noise	Slew	Trajectory
Calm (F1)	0	0	0.05	90	Circular
UAV (F2)	4	8	0.2	90	Circular
Stress (F3)	7	15	0.35	120	Random

7.2 Live disturbance and plant parameters

Table 8: Configurable live parameters

Parameter	UI / keys	Notes
Turbulence (Cn2 proxy)	Slider; Q/A	Maps to atmospheric wander/scintillation.
Vibration	Slider; W/S	Platform tip/tilt on FOV.
Sensor noise	Slider; E/D	Image grain; interacts with weather.
Slew rate	Slider	Camera plant authority.
Trajectory	Buttons; 1–4	Includes PS-required figure-8.
Detector mode	Buttons; T	Classical vs AI (hybrid).
Beacon count	Buttons; B	1 or 2.
Noise type	N	Legacy / Gaussian / salt-pepper / Poisson.
Weather grade	M	Clear → Haze → Fog → Rain → Low light.

7.3 Keyboard reference

Table 9: Complete keyboard shortcuts

Key	Action
1 / 2 / 3 / 4	Trajectory: linear / circular / random / figure-8
Q / A	Turbulence + / -
W / S	Vibration + / -
E / D	Sensor noise + / -
T	Toggle Classical ↔ AI
B	Toggle 1 ↔ 2 beacons
N	Cycle noise type
M	Cycle weather preset
L	Blind detector for a fixed frame count (force re-acq test)
F1 / F2 / F3	Calm / UAV / Stress
ESC	Quit application

8 Logging and Performance Report

8.1 Automatic logs

Each interactive session writes:

- logs/run_YYYY-MM-DD_HHMM.csv — per-frame / session metrics;
- logs/run_*.summary.json — run sheet (acquisition time, lock %, FPS, errors, etc.).

8.2 Streamlit report

```
streamlit run report.py
```

1. Select a CSV from logs/.
2. Optionally enable comparison of two runs (Classical vs AI).
3. Download text or PDF performance report for judges.

8.3 Matched Classical vs AI benchmark

```
python scripts/run_comparison.py --duration 60
```

Produces matched-seed CSV pairs under logs/ for fair detector comparison.

8.4 Benchmark-2 (judge MP4, PTZ bypass)

```
python scripts/generate_ps_test_video.py
python scripts/benchmark_video.py --video logs/benchmark2/ps_sample.mp4 --gt logs/
benchmark2/ps_sample_gt.csv
```

Use the same script on judge-provided videos; set `--detector ai` when needed. Outputs include centroid CSV and a summary JSON with `ptz_bypassed: true`.

8.5 Other evaluation utilities

- Acquisition Monte Carlo: `scripts/monte_carlo_acquisition.py`
- Detection Pd/Pfa: `scripts/evaluate_detection.py`
- Demo video capture: `scripts/record_demo_video.py`

9 Troubleshooting

Table 10: Common issues

Symptom	Action
AI mode unavailable / reverts	Ensure <code>weights/beacon_yolov8n.pt</code> exists; retrain if missing.
Low FPS with AI on CPU	Expected; use Classical for smooth live demos or enable CUDA.
Black / empty FOV	Reduce Stress disturbances; wait for re-acquisition; check beacon in overview.
No log files	Run longer than ~ 1 s before ESC; check write permission on <code>logs/</code> .
Import / package errors	Activate <code>.venv</code> and re-run <code>pip install -r requirements.txt</code> .
PS window too large	Use a high-resolution display or stay on Physics profile for demos.

10 Related Documents

- Technical report (algorithms, architecture, results): `docs/sih_technical_report.pdf`
- Compliance notes: `docs/sih_compliance.md`, `docs/ps_compliance_gaps.md`
- Markdown quick reference (this manual’s source twin): `docs/user_manual.md`

End of User Manual — BeamLock / Team Cadets / SIH26169.